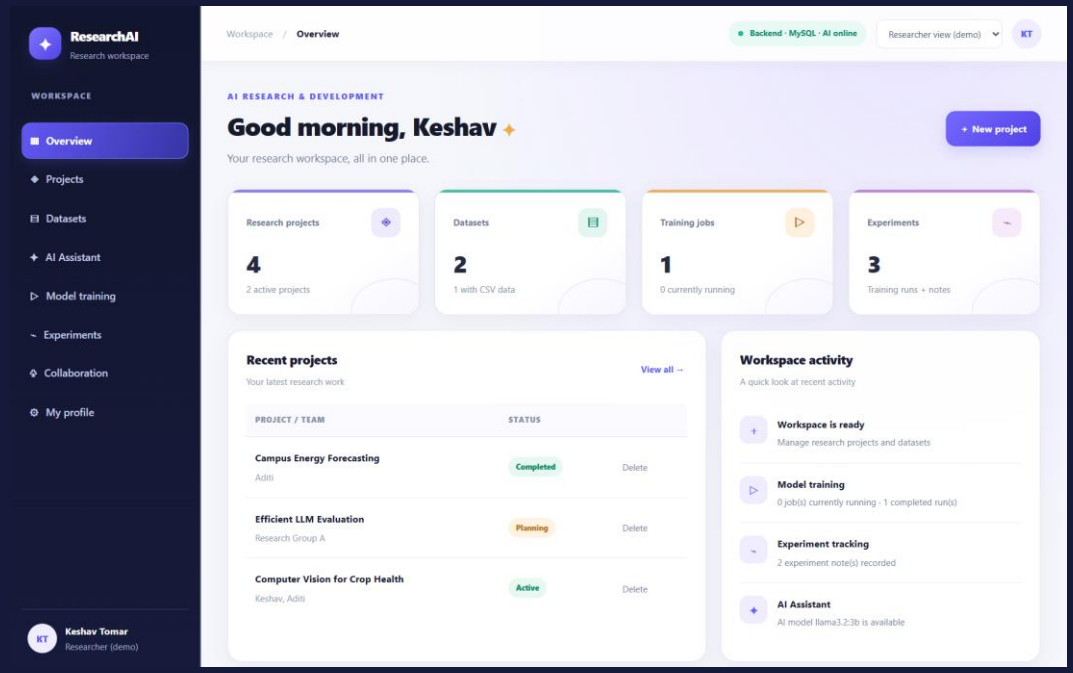
GUVI PROJECT · REVIEW 1

ResearchAI

AI Research & Development Platform

Manage research projects and datasets, train a real ML model, and get help from an AI assistant.



Keshav Tomar

Live: fantastic-pavlova-37ef51.netlify.app

Code: github.com/keshavtomar1606-gif/AI_Research_Plateform

WHY THIS PROJECT

Problem & objectives

The problem

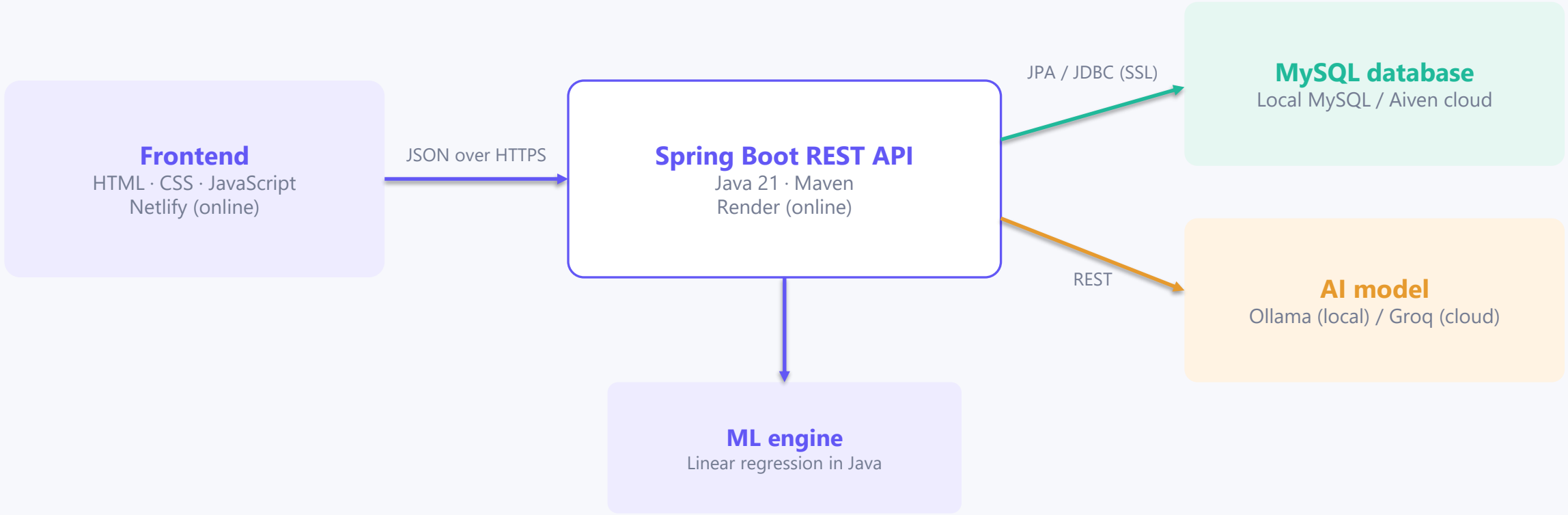
- **Scattered work** projects, datasets and results live in different files and chats
- **No reproducibility** ML results are hard to repeat or compare
- **Expensive AI tools** students need help without paid API subscriptions
- **Fake progress** many demos only show a progress bar, not real training

What ResearchAI delivers

- **One dashboard** for projects, datasets, training and experiments
- **MySQL database** every project, dataset and run is saved (full CRUD)
- **Real ML training** linear regression with train/test split, MAE, RMSE, R^2
- **AI assistant** free: local Ollama, or Groq online; history is saved
- **Responsive, safe UI** works on phones; inputs validated, HTML escaped

HOW IT FITS TOGETHER

System architecture & tech stack



Java 21 · Spring Boot 3.4 · Spring Data JPA · MySQL 8 · HTML/CSS/JS · Ollama / Groq · JUnit + Playwright · Docker

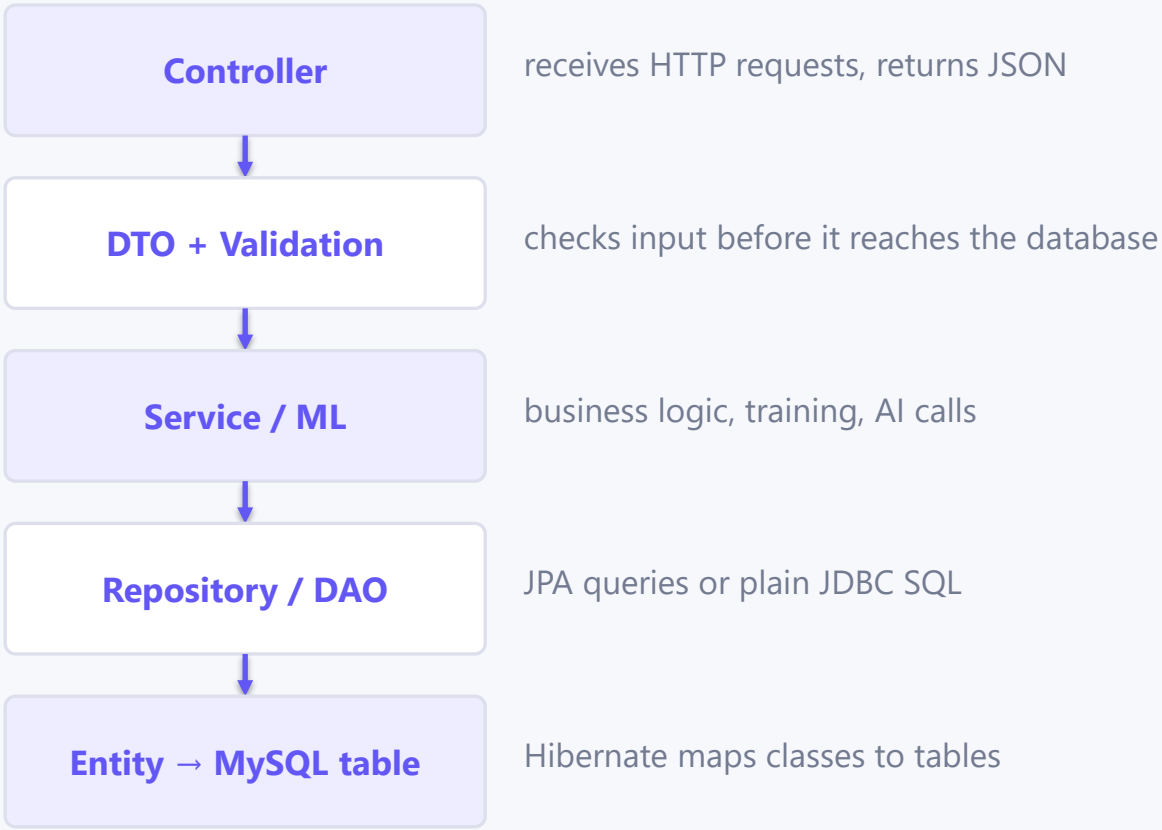
Secrets (DB password, API key) are read from environment variables, never stored in code.

ORGANISED AND MODULAR

Project structure

```
AI_Research_Platform/
├─ frontend/          UI (index.html, app.js, style.css)
├─ backend/           Spring Boot API
│  └─ src/main/java/com/aiplatform/
│     ├─ controller/  REST endpoints
│     ├─ dto/         request/response + validation
│     ├─ service/     AI providers, rate limit
│     ├─ ml/          CSV, regression, metrics
│     ├─ repository/  JPA database access
│     ├─ dao/         plain JDBC database access
│     ├─ model/       entities (tables)
│     ├─ exception/   custom exception classes
│     ├─ web/         ExportServlet (HttpServlet)
│     └─ config/      CORS, security, errors
├─ database/schema.sql
├─ scripts/  · docs/
└─ README.md · DEPLOY.md
```

Layered backend (each layer has one job)



MYSQL SCHEMA (4 TABLES)

Database design

1 dataset → many runs

research_project

```
PK id          BIGINT
   title       VARCHAR NOT NULL
   description VARCHAR(2000)
   team        VARCHAR
   status      VARCHAR
```

experiment (plain JDBC)

```
PK id          BIGINT AUTO_INCREMENT
   name        VARCHAR NOT NULL
   model       VARCHAR NOT NULL
   metric      VARCHAR
   status      VARCHAR
   created_at  DATETIME
```

dataset

```
PK id          BIGINT
   name        VARCHAR NOT NULL
   owner       VARCHAR
   file_name   VARCHAR
   status      VARCHAR
   csv_content LONGTEXT
   column_names VARCHAR
   row_count   INT
   created_at  DATETIME
```

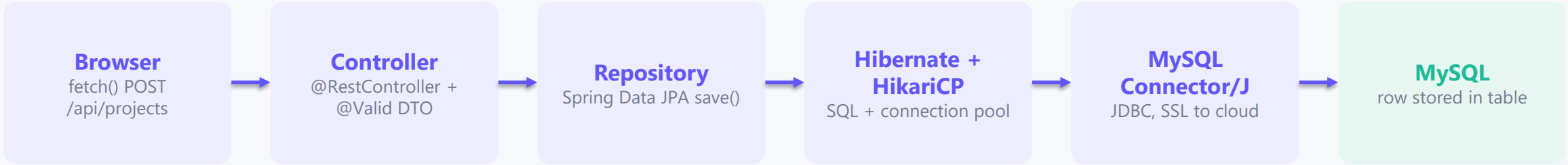
training_job

```
PK id          BIGINT
FK dataset_id  BIGINT
   job_type    ML_TRAINING/RECORD
   status      Queued…Completed
   progress    INT
   target_column VARCHAR
   feature_columns VARCHAR
   mae / rmse / r2 DOUBLE
   coefficients_json VARCHAR
   started_at / finished_at DATETIME
```

- **3 tables via JPA/Hibernate, 1 via plain JDBC** (same MySQL database)
- **Validation** NOT NULL, length limits, allowed status values
- **Results stored** metrics + learned coefficients per run
- **Link** training_job.dataset_id → dataset.id (logical key: runs keep results if a dataset is deleted)
- **Script** database/schema.sql (verified against the code)

FROM BUTTON CLICK TO MYSQL ROW

Database connectivity



Configuration (no passwords in code)

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
hikari.maximum-pool-size=5
```

Locally: backend/config/local.properties (git-ignored).
Online: Render environment variables.

Proof: health check

```
GET /api/health → "database":"UP", "ai":"UP"
```

- **Two styles** JPA repositories + plain JDBC DAO (PreparedStatement, transactions)
- **Tested** create → read → update → delete against real MySQL
- **Errors** DB down shows a clear message, never a fake “saved”

WHERE EACH CONCEPT IS IMPLEMENTED

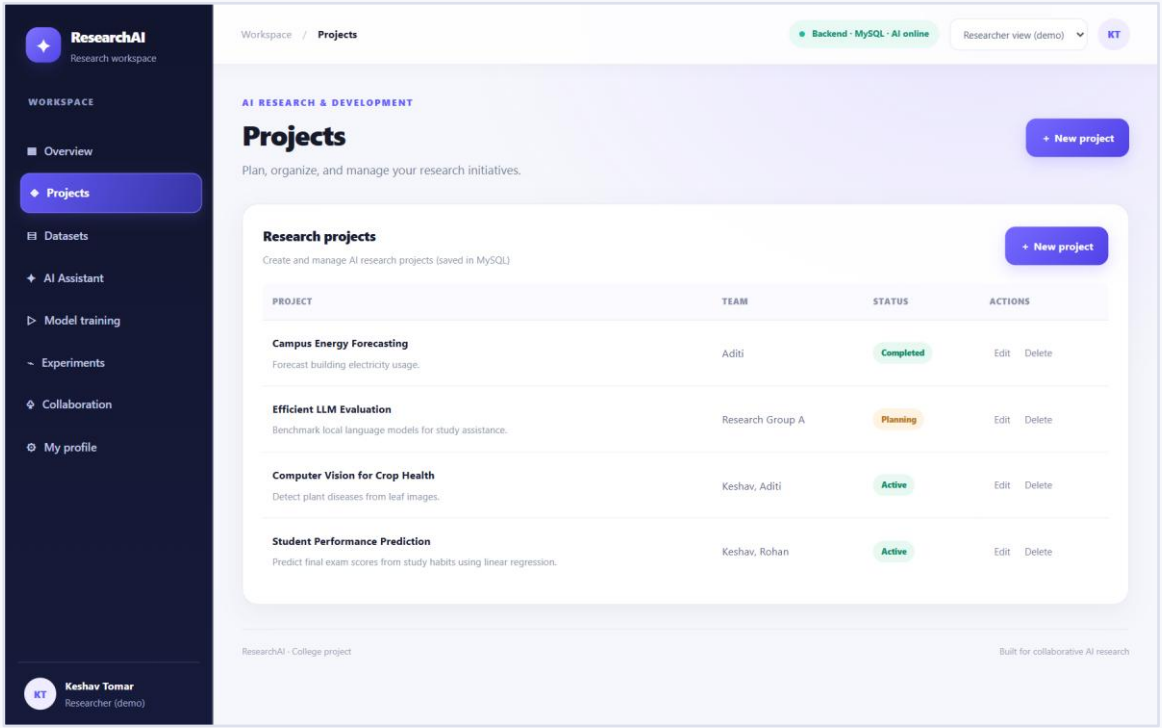
Core Java concepts in the code

Interfaces	ChatProvider · ExperimentDao · JPA repositories
Inheritance	ResearchAiException → TrainingDataException, NotFoundException, DaoException · ExportServlet extends HttpServlet
Polymorphism	Ollama / Groq providers behind one interface (chosen at runtime) · each exception overrides status()
Exception handling	Custom hierarchy · GlobalExceptionHandler (clean JSON errors) · try-with-resources
Collections & generics	List, Map, Set, Deque, Optional · ConcurrentHashMap · JpaRepository<T, ID>
Multithreading	ThreadPoolExecutor runs training jobs · synchronized rate limiter · volatile cache
JDBC	JdbcExperimentDao: Connection → PreparedStatement → ResultSet · generated keys · commit / rollback
Servlets & web	ExportServlet.doGet() streams CSV · AccessKeyFilter (servlet filter) · REST controllers

Packages: controller · service · ml · dao · repository · model · exception · web · config — 44 automated tests

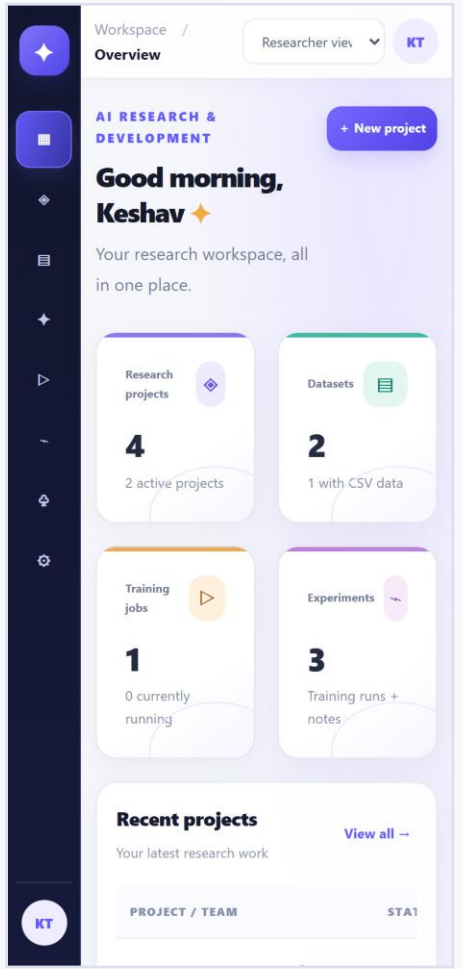
AESTHETICS AND USABILITY

UI design & responsiveness

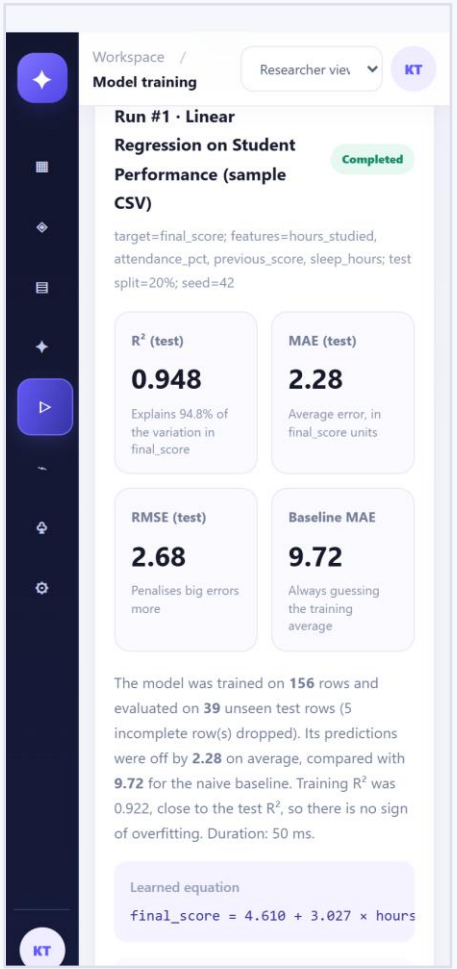


Desktop: Projects page (data loaded from MySQL)

- **Consistent theme** purple palette, cards
- **Clear status** coloured tags
- **Feedback** loading, success/error
- **Safe forms** Cancel never saves
- **Responsive** icon sidebar on phones
- **Accessible** labels, keyboard support



Phone: Overview



Phone: Training result

WHERE THE PROJECT STANDS

Progress, testing & next steps

44
backend tests passing

49
browser test checks passing

0.95
R^2 on sample test data

Live
Netlify + Render + Aiven + Groq

Completed for Review 1

- **Project structure** modular frontend + layered backend
- **Database design** 4 tables, schema script, validation
- **Database connectivity** JPA + plain JDBC DAO, local and cloud
- **Servlet** ExportServlet for CSV downloads
- **UI/UX** consistent design, responsive layout
- **Extra** real ML training and AI assistant working

Next steps

- **Real login** Spring Security with hashed passwords
- **Admin pages** users & resources stored in MySQL
- **More models** classification, model comparison
- **Charts** visualise predictions vs actual values

Live demo: fantastic-pavlova-37ef51.netlify.app · Code: github.com/keshavtomar1606-gif/AI_Research_Plateform